The Business Brain

Layered Architecture & Build Manual

Every layer, every workflow, how they connect — and how to build each one

One orchestrating intelligence across all business functions

Source-agnostic by design · TranzAct is the first adapter

Author: Ayush Mangal

Companion to: The Business Brain (concept) & Workflow Reference

Table of Contents

Part 0 — The Frame

0.1 What we are building

We are building one **brain** that runs a business, not another toolbox that records it. Today most companies run on software that logs what already happened — an ERP logs transactions, a CRM logs contacts, a helpdesk logs tickets — and a human is the connecting intelligence stitching the drawers together by hand. The Business Brain inverts that: a single intelligence perceives across every function, reasons about what to do, and acts, keeping a human in the loop exactly where judgement and money are at stake. The tools become sources the brain reads from and writes to, not islands a person ferries between.

This document is the engineering build manual for that brain. It walks every layer of the system from the ground up: what the layer is, what we have already built, what to improve, what is still to build, the actual modules and code with the reason for each, how the layer connects to the ones around it, and which business workflows depend on it. It then maps all thirteen business fields and every workflow inside them onto these layers, and shows how they wire together into a single connected system.

> **One important framing for the whole document**
> The current product serves **TranzAct data only**, but nothing in the architecture is TranzAct-specific. TranzAct is simply the *first adapter*. Every layer above the adapter speaks a source-independent canonical language, so adding Tally, Busy, email or any API later is a new adapter and zero changes to the brain. Read every layer below as source-agnostic.

0.2 The one idea everything is built from: the universal loop

Every workflow in this product — chasing an overdue invoice, reordering stock, drafting a support reply, flagging a risky contract — is the same five-beat loop pointed at different data. Internalising this one shape makes the entire system easy to reason about.

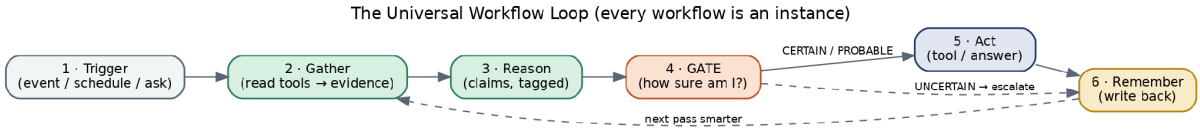

Figure 1 — The universal loop. Every workflow is one instance of it.

1. **Trigger** — an event (a payment lands), a schedule (3 a.m. sync), or a person asking a question.
2. **Gather** — the brain reads the data it needs, through tools, turning raw data into tagged evidence.
3. **Reason** — it builds claims, deciding what the data means.
4. **Gate** — it checks how sure it is. This is the heart of the design (see 0.4).
5. **Act** — it does the thing (or answers), within what the gate allows.
6. **Remember** — it writes the outcome back to memory so the next pass is better informed.

0.3 The layer model

The brain is a stack of layers. Each has one responsibility and one strict rule about what it is allowed to know. This separation is the entire reason a new data source or a new workflow can be added without disturbing the rest. The figure shows the full stack and how much of it exists today.

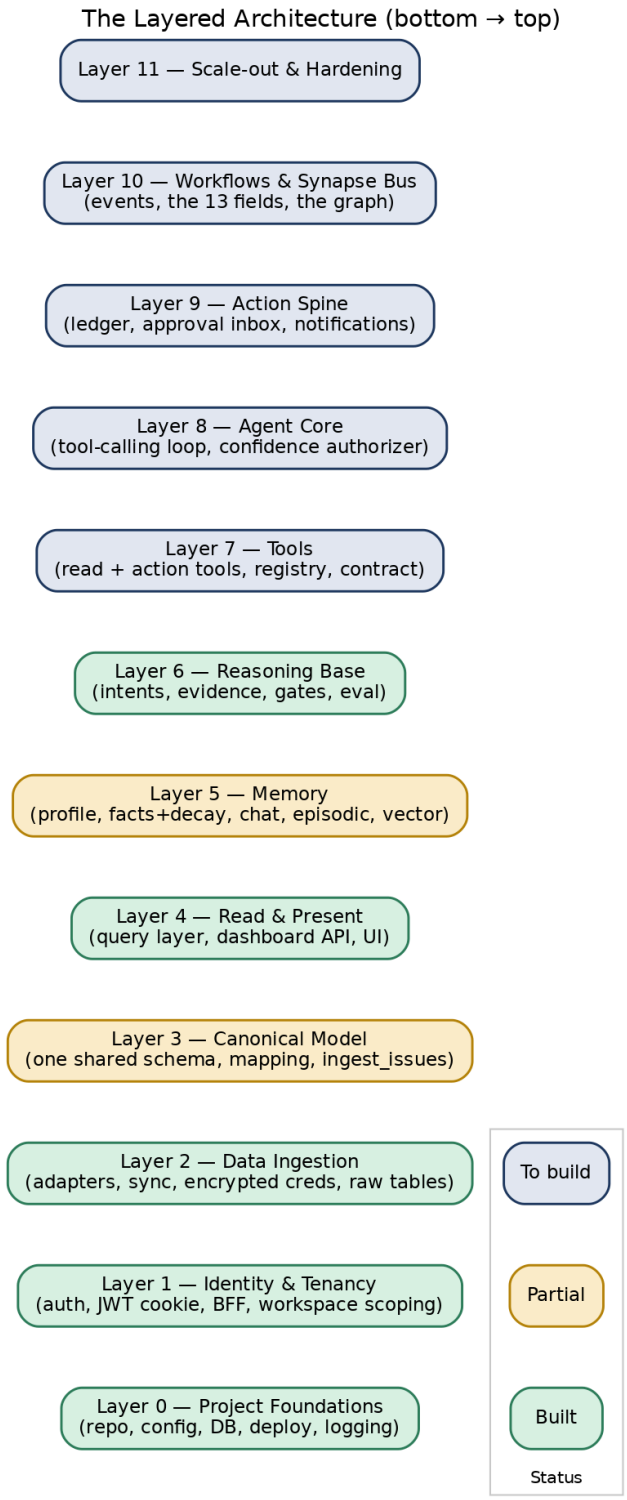

Figure 2 — The layered architecture, ground-up, with current status.

The lower layers (the substrate) are the hard, defensible part — adapters into messy local systems, a canonical model, memory, and a confidence-gated reasoning core. The upper layers (tools, agent, actions, workflows) are the apps that run on that substrate. We have built the substrate and the safe reasoning core; the build ahead is the action half.

0.4 The confidence gate — what makes autonomy safe

Business decisions move real money, so a confidently wrong answer is worse than no answer. Every workflow therefore passes through a three-way gate before it acts: **CERTAIN** actions run automatically; **PROBABLE** actions run only after a human confirms; **UNCERTAIN** actions escalate to a person. One unbreakable rule overrides confidence entirely: anything that moves money or files with a regulator always needs a human, no matter how sure the brain is.

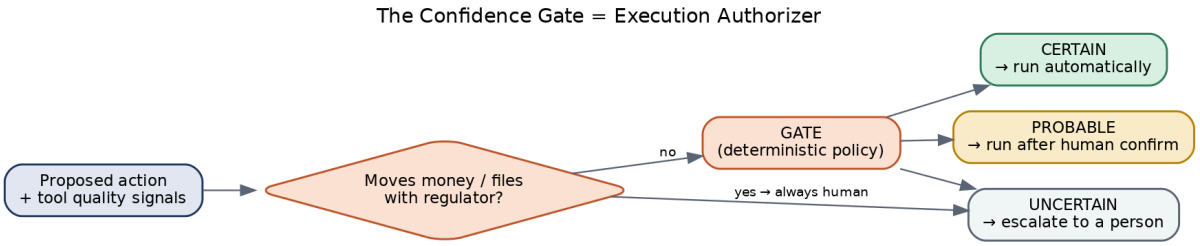

Figure 3 — The confidence gate as the execution authorizer.

> **The design principle that makes the AI trustworthy**
> The model **proposes**; deterministic code **disposes**. The AI may decide what to do, but whether it is safe to actually do it is decided by a rules engine we own — never by the model grading itself. Confidence is computed from real signals (is the data fresh? did the match resolve unambiguously? is this a money action?), not from the model saying 'I'm 90% sure'. This is the line between a controlled brain and a reckless agent.

0.5 Status at a glance

Layer	What it is	Status
0 Foundations	Repo, config, DB, deploy, logging	BUILT
1 Identity & tenancy	Auth, JWT cookie, BFF, per-brand scoping	BUILT
2 Data ingestion	Adapters, sync, encrypted creds, raw tables	BUILT
3 Canonical model	One shared schema; mapping; ingest issues	PARTIAL
4 Read & present	Query layer, dashboard API, UI	BUILT
5 Memory	Profile, facts+decay, chat, episodic, vector	PARTIAL
6 Reasoning base	Intents, evidence, gates, eval	BUILT
7 Tools	Read + action tools, registry, contract	TO BUILD

Layer	What it is	Status
8 Agent core	Tool-calling loop, confidence authorizer	**TO BUILD**
9 Action spine	Ledger, approval inbox, notifications	**TO BUILD**
10 Workflows & synapse bus	Events, the 13 fields, the graph	**TO BUILD**
11 Scale-out	More adapters/verticals, hardening	**TO BUILD**

Part I — The Layers, Built One by One

Each layer below follows the same template: **what it is**, **what we have**, **what to improve / build**, **the modules and code** (with the reason for each), **how it connects**, and **the workflows it powers**.

Layer 0 — Project Foundations

Status: BUILT

What it is

The skeleton everything runs on: how the code is organised, how configuration and secrets are loaded, how we reach the database, how we deploy, and how we see what is happening (logging).

What we have

- **Monorepo** with two deployable services — a Python FastAPI backend (the brain) and a Next.js frontend (the face) — plus a shared TypeScript types package.
- **Configuration** centralised in `config.py`, read from environment variables via `python-dotenv`. No secret is ever hard-coded.
- **Database access** through a single connection helper to PostgreSQL.
- **Deployment** as a Docker container (Dockerfile, railway.json, apprunner.yaml) running uvicorn; frontend on Vercel.

Why this shape: two services, not one, because the brain (Python, data + AI) and the face (TypeScript, UI) have different runtimes and scaling needs. A monorepo keeps them versioned together and lets them share types.

What to improve / build

- A proper CI pipeline (run the eval suite + tests on every push) PARTIAL
- Structured, queryable logging and request tracing (today logs are plain text) PARTIAL

How it connects

Every other layer depends on this one for config, the DB connection, and deployment. Nothing here knows about the business — it is pure plumbing.

Layer 1 — Identity & Tenancy

Status: BUILT

What it is

Who the user is (authentication), what they are allowed to touch (authorisation), and how one account can run several brands without their data ever mixing (multi-tenancy). It is built first because every row and every request in the whole system is scoped by user and brand.

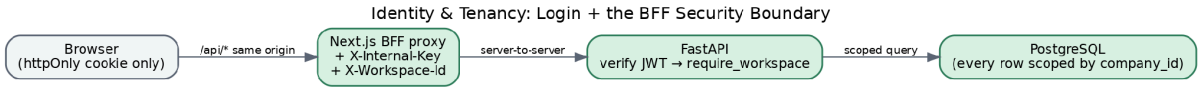

Figure 4 — Login and the BFF security boundary.

What we have

Authentication. A user logs in with email + password. The password is checked against a bcrypt hash in the `users` table (bcrypt is a deliberately slow, salted hash, so stolen hashes can't be brute-forced). On success the backend issues a **JWT** — a signed token carrying the user's email and brand — and sets it as an `httpOnly`, `Secure`, `SameSite=Strict` cookie.

```
# api/routes/auth.py  (the essence)
def login(req, response):
    row = db("SELECT password_hash, company_id FROM users WHERE email=%s", req.email)
    if not row or not bcrypt.verify(req.password, row.password_hash):
        raise HTTPException(401, "Invalid email or password")
    token = jwt.encode({"sub": req.email, "company_id": row.company_id,
                        "exp": now()+8h}, JWT_SECRET, "HS256")
    response.set_cookie("vb_access_token", token,
        httponly=True, secure=True, samesite="strict")   # invisible to JS
```

Why a JWT in an httpOnly cookie (not localStorage): an httpOnly cookie cannot be read by JavaScript, so a cross-site-scripting bug cannot steal the session; Secure means HTTPS-only; SameSite=Strict blocks CSRF. A token in localStorage has none of these protections. The JWT is **ours**, issued by our backend — we do not use the database host's auth system at all.

The BFF (Backend-for-Frontend) boundary. The browser never calls the backend directly. It calls Next.js API routes on the same origin, which forward the request server-to-server, adding an `X-Internal-Key` (so the backend rejects anyone who bypasses the proxy) and the `X-Workspace-Id` header. The backend URL is a server-only secret, never shipped to the browser.

Multi-tenancy. A 'workspace' is one brand = one source account = one data scope. Every data table carries a `company_id` column, and a `require_workspace` dependency resolves and authorises the brand before any query runs.

```
# api/routes/workspaces.py  (the gatekeeper every endpoint depends on)
def require_workspace(request, user=Depends(get_current_user)) -> str:
    ws = request.headers["X-Workspace-Id"] or user.company_id
    if not db("SELECT 1 FROM companies WHERE id=%s AND (owner_id IS NULL OR owner_id=%s)",
              ws, user.sub):
        raise HTTPException(403, f"No access to workspace {ws}")
    return ws        # every query is then scoped: ... WHERE company_id = ws
```

What to improve / build

- Roles (owner / admin / member / viewer) and — crucially for the brain — **who is allowed to approve money actions** TO BUILD
- Team invites, password reset, email verification TO BUILD

- Profile & preferences (name, notification settings) **PARTIAL**
- Optional Google sign-in / 2-factor for larger customers **TO BUILD**

How it connects

Sits under everything: the BFF wraps the frontend (Layer 4), `company_id` scopes ingestion (Layer 2), the cache (Layer 3), queries (Layer 4), memory (Layer 5) and every workflow (Layer 10). Roles will gate the approval inbox (Layer 9).

Workflows it powers

Indirectly, all of them — every workflow runs inside a brand and is gated by who the user is. Directly, the People-Management workflows (onboarding/access) and any action requiring an approver depend on the roles added here.

Layer 2 — Data Ingestion (Adapters & Sync)

Status: BUILT

What it is

The plugs that pull data out of the messy real-world systems (TranzAct today; Tally, Busy, email, APIs later) and land it in our database. This is the part nobody enjoys building — which is exactly why it is part of the moat.

What we have

The adapter contract. Every source implements the same interface — log in, fetch, hand back rows — so the layers above never care which source the data came from.

The TranzAct adapter, in two modules:

- `auth.py` — logs in, caches the bearer token in memory keyed per account, reads the token's real expiry from the JWT so it refreshes at exactly the right time, and tries a cheap refresh before a full re-login.
- `client.py` — the single `fetch_report()` that walks pages until the report is exhausted, throttles to stay under the source's rate limit, backs off on errors, and supports resumable chunks so a long sync survives a restart.

Credential storage. Source credentials are encrypted with Fernet (authenticated AES) before being written to the DB and decrypted only in memory at sync time — never returned by any API.

The pipeline + scheduler. A `BasePipeline` gives every report the same safe contract:

```
# pipelines/base.py  (the contract every report inherits)
def run_chunk(self, company_id, creds, ...):
    run_id = log_start(tz_sync_runs, "running")
    raw   = fetch_report(self.REPORT_ID, creds=creds, ...)   # adapter
    rows  = self._dedupe(self._validate(raw))                # Pydantic; bad rows skipped
    self._upsert(conn, rows, company_id)                     # ON CONFLICT DO UPDATE
    log_finish(run_id, "success", fetched=len(raw), upserted=len(rows))
    # on any exception -> log "failed" and LEAVE OLD DATA IN PLACE
```

> **Why UPSERT, and why never truncate-and-reload**
> The source has no usable server-side date filter, so syncs re-fetch overlapping rows. Keying the write on a stable content hash means a re-fetched row overwrites in place instead of duplicating. And updating in place rather than wiping-and-reloading means a sync that dies halfway never empties the table — the dashboard keeps serving the last good data. 'Stale is better than empty' is enforced right here at the database.

Orchestration. APScheduler runs all reports hourly, staggered by minute so they never burst against the rate limit, once per connected brand. A heavier full historical sync walks every page with a resumable cursor.

What to improve / build

- More adapters: Tally, Busy, email/IMAP, generic CSV/API **TO BUILD**
- Real-time **webhook ingestion** so events (a payment, a new ticket) push in instantly instead of waiting for the hourly pull — the trigger source for the event-driven brain **TO BUILD**
- A connector test + health surface per source **PARTIAL**

How it connects

Writes raw rows into the cache (Layer 3’s source tables). Triggered by the scheduler (and later the event bus, Layer 10). Uses encrypted credentials stored via Layer 1. Knows nothing above it.

Workflows it powers

Every workflow that reads business data — which is nearly all of them — depends on this layer having pulled that data in. The Technology field’s ‘data-freshness watchdog’ workflow reads this layer’s sync logs directly.

Layer 3 — The Canonical Model

Status: PARTIAL

What it is

The one normalised language every part of the brain speaks. The source tables are shaped like TranzAct; the canonical tables are shaped like a *business* — Customer, Invoice, SKU, Payment, GL — regardless of which system the data came from. This is the moment the product stops being ‘a TranzAct tool’ and becomes ‘an engine that currently has one adapter’.

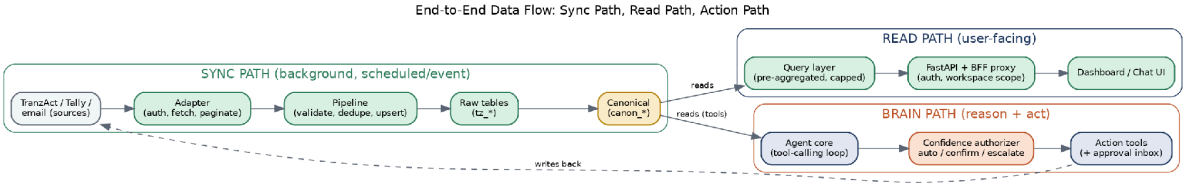

Figure 5 — Data flows left-to-right: source → adapter → raw → canonical → consumed by the read path and the brain.

What we have

- A `SourceAdapter` contract: `extract → map → load`, where `map` returns either a clean canonical row or an `Unmapped` record.
- The first mapper (TranzAct → canonical) building Customers, Sales Invoices + lines, Inventory Items and AR/Payments, re-run after every sync.
- Idempotent upserts keyed on `(company_id, source, source_ref)`, so re-running refreshes in place.

> **Two rules that make this layer trustworthy**
> **Always keep the raw row** — so we can re-map later without re-fetching. And **log, don’t guess** — anything that can’t be confidently mapped goes to an `ingest_issues` table, never a plausible fake value and never a silent drop. The brain is only as honest as its data, so bad data is surfaced, not invented.

What to improve / build

- Extend the canonical schema to the full set the workflows need: GL accounts, Purchase docs, Orders, Production, Tickets, Contracts, Employees PARTIAL
- **Entity resolution** — the same customer arriving from two sources (or spelled two ways) must resolve to one canonical entity TO BUILD
- A second adapter (Tally) to prove the model is genuinely source-independent TO BUILD

How it connects

Fed by Layer 2 (raw tables). Read by everything above: the query layer (Layer 4), the tools (Layer 7) and the agent (Layer 8) all read **canonical, never source** tables — which is what insulates the brain from any one system’s quirks.

Workflows it powers

All cross-functional workflows — the synapses — only work because every field reads the same canonical Customer and Invoice. A churn score fusing support + payments + engagement is impossible unless all three resolve to one canonical customer.

Layer 4 — Read & Present (Query Layer + UI)

Status: BUILT

What it is

The layer that turns canonical data into answers a human can see — the pre-aggregated query functions, the API that serves them, and the dashboard UI. It delivers value on its own, and it doubles as the foundation the AI reuses: the same query functions become the AI's read-tools in Layer 7.

What we have

- ~45 query functions (`queries.py`) that run SQL aggregation and return **typed dictionaries, never raw rows**, with built-in Top-N caps (top 15 customers, 20 SKUs, etc.).
- ~35 dashboard endpoints, each returning a consistent envelope with a freshness flag.
- A Next.js dashboard (revenue, customers, AR, inventory, purchases, production, orders, …) fetched with SWR, every panel showing how fresh its data is.

```
{ "data": { ... },                              # the answer
  "meta": { "last_synced_at": "...Z",           # always show data age
            "report_id": 29, "stale": false } }  # stale if last sync > 25h
```

> **Why pre-aggregate into typed dicts (the anti-hallucination foundation)**
> An LLM handed thousands of raw rows hallucinates. So the AI is never given raw rows — it is given the output of these functions: a small, named, capped summary. The function does the heavy lifting; the model only reasons. Building this for the dashboard **also** builds the safe data surface the brain will read. One investment, two payoffs.

What to improve / build

- Finish the onboarding flow (connect source → set business profile → choose how much history to sync) PARTIAL
- New UI surfaces the brain needs: the **approval inbox**, a notifications/activity feed, and the brain graph view TO BUILD

How it connects

Reads Layer 3 (canonical). Served through Layer 1 (BFF + scoping). Its query functions are promoted into tools in Layer 7. The UI gains the action surfaces from Layer 9.

Workflows it powers

Directly powers the read-only workflows that exist today — most importantly Finance's **'natural-language financial querying'**, which is the single workflow the current product already ships. Every other field's dashboards live here too.

Layer 5 — Memory

Status: PARTIAL

What it is

Everything the brain remembers between one moment and the next. There are four distinct kinds, and conflating them causes bugs, so we keep them separate.

Kind of memory	What it holds	Status
Business profile	Static facts set at onboarding: industry, fiscal year, healthy-margin benchmark, seasonality.	BUILT
Durable facts	Things the owner teaches (‘Customer X pays in 7 days’); supersede + decay.	BUILT
Conversation memory	Chat threads + turns, so a tab can be reopened.	PARTIAL
Episodic / event log	A record of what the brain did each loop, so it learns and is auditable.	TO BUILD
Semantic (vector) memory	Searchable docs, tickets, contracts, KB — for retrieval workflows.	TO BUILD

What we have

Durable facts that supersede and decay. When the owner confirms something it is saved as a structured fact; a new value retires the old one so there is always exactly one current truth; and facts go **stale** on two signals — a time expiry, or a contradiction with live data (a stated 7-day term when invoices are 60 days overdue). A stale fact is flagged, not deleted, so the brain *re-asks* instead of confidently repeating something now-wrong.

Conversation storage. Each Ask turn (question + full structured answer) is saved into a thread, scoped to the (user, brand) pair, and threads auto-name from the first question — so reopening a tab restores the whole conversation.

What to improve / build

> **The honest gap: chat history is stored but not yet fed back**
> Today each new question is answered fresh — the past conversation is **displayed** but not **fed back into the model as context**. So a follow-up like ‘and which of those are overdue?’ doesn’t yet know what ‘those’ means. For an agentic brain that takes multi-step actions, feeding recent turns back into the model is essential, and it is the first memory item to build.

- Feed recent conversation turns into the agent’s context (multi-turn) TO BUILD
- **Episodic log** — persist every brain loop (trigger, tools called, decision, gate result, action) for learning + audit TO BUILD
- **Vector store** — embed documents/tickets/contracts so retrieval workflows (support deflection, legal Q&A, R&D mining) can search by meaning TO BUILD

How it connects

Read by the reasoning core (Layer 6) and the agent (Layer 8) as context before answering; written back to after each loop (the ‘Remember’ beat). The vector store is read by retrieval tools (Layer 7).

Workflows it powers

Vector memory powers Support tier-1 deflection, Legal document Q&A and R&D customer-need mining. Durable facts power Finance’s payment-stretch and credit reasoning. The episodic log powers every workflow’s ‘learns over time’ property and the audit trail.

Layer 6 — The Reasoning Base

Status: BUILT

What it is

The safe, deterministic core that answers a question with calibrated confidence — and the place where ‘say no’ is made structural rather than a polite request to the model. It is the proving ground for the grounding discipline before the brain is ever allowed to take an action.

What we have — the three-stage pipeline

```
retrieve (deterministic)  ->  reason  ->  validate (deterministic)
```

- **Retrieve** — call the query functions; each number becomes a tagged piece of `Evidence` with a stable id.
- **Reason** — build `Claims`, each tagged `computed` / `inference` / `unknown`. A computed claim MUST cite an evidence id.
- **Validate** — every computed claim is checked to trace to real evidence; if it can’t, it is downgraded so it can never masquerade as a proven fact.

Confidence gates. Three checks set the label: Data (is there evidence?), Rule (did we recognise the question?), Confidence (is the conclusion a computation or a judgement?). An unanswerable question — margin with no cost data, a forecast with no order book — deliberately refuses and returns UNCERTAIN.

The Claude layer, tightly boxed. When enabled, the model does only two things: route an unrecognised question to one known intent, and **rephrase** the validated claims into consultant-quality prose. It is handed the claims, never the database. A numeric guard then scans the prose and rejects any rupee figure that wasn’t computed, with one self-correcting retry before falling back to the safe template.

The eval harness. Because an LLM feature has no compiler, the eval suite is the compiler: golden cases across answerable / must-refuse / memory / must-not-say, scored on metrics that **ship-block** the build if the engine ever states an unsupported number or utters a banned phrase.

What to improve / build

- This base stays — but in Layer 8 its hard-coded keyword router is replaced by tool-calling, and its answer-confidence gate is generalised into an action authorizer. The **evidence → validate → gate** spine is reused intact. PARTIAL

How it connects

Reads Layers 4/5 (queries + memory). Today it is the engine behind the Ask endpoint. Tomorrow it becomes the ‘Reason + Gate’ beats inside the agent core (Layer 8).

Workflows it powers

It is the live brain behind every ‘ask the business a question’ workflow across all fields — and the safety pattern (grounded claims, calibrated refusal) that every action workflow will inherit.

Layer 7 — Tools (the first all-new layer)

Status: TO BUILD

What it is — explained from scratch

A **tool** is just a function the AI is allowed to call, described in a way the model understands. You give the model a menu of tools, each with a name, a description, and a typed list of inputs; the model reads the menu and replies 'call `get_ar_summary` with company_id=kbrushes'; your code runs the real function and hands back the result. That is the entire idea behind 'tool calling' and behind MCP — there is nothing mystical about it.

> **MCP vs in-process tools — what to use and why**
>
> **MCP (Model Context Protocol)** is a standard way to expose tools as a separate service that any AI client can connect to. It shines when tools must be shared across many apps or run in another process/language. **In-process tools** are just functions registered in the same backend, passed to the model via the Anthropic SDK's native tool-use.
>
> **Recommendation: start with in-process tools using the Anthropic SDK you already use.** Your tools are Python functions that already live next to your data and your gate. Wrapping them as a separate MCP server adds a network hop and ops for no benefit yet. Adopt MCP later *if* you want third-party clients (or other internal apps) to reuse the same tools. The tool **contract** below is designed so either path works without rewriting the tools.

The tool contract (the key new abstraction)

Every tool — read or action — declares the same metadata, so the agent and the gate can reason about it uniformly:

```
@tool(
  name="create_reorder",
  description="Raise a draft purchase order for a SKU below its reorder point.",
  inputs={"sku_code": str, "quantity": int, "vendor_code": str},
  side_effect="writes",          # one of: read | writes | moves_money | files_regulator
  gate="action",                  # 'read' tools never need approval; 'action' tools do
)
def create_reorder(ctx, sku_code, quantity, vendor_code) -> ToolResult:
    ...
    return ToolResult(
        data={"po_id": "...", "value": 84000},
        evidence=[Evidence("po_value", "PO value", 84000, "Rs 84,000")],
        quality={"vendor_known": True, "data_fresh": True})  # signals the GATE reads
```

Why each field matters: the `side_effect` class is what the gate uses to force money/regulator actions to a human; `evidence` keeps the grounding guarantee (the model may only state values a tool returned); `quality` signals are what let the gate compute confidence from facts instead of the model's opinion.

What to build

- **A tool registry** — a catalogue the agent is handed, filtered by the user's permissions and the workflow.
- **Read tools** — wrap the existing query functions (cheap; they already return typed evidence). PARTIAL

- **Action tools** — draft_collection_email, create_reorder, post_journal_entry (money-gated), draft_reply, create_ticket, generate_nda, … one per action a workflow needs.
- **An execution runtime** — runs the tool, captures evidence + quality signals + a dry-run preview, and records it to the action ledger (Layer 9).

How it connects

Read tools call Layer 4’s query functions over Layer 3’s canonical data. Action tools call out to the world (email, source write-back) and record to Layer 9. The registry is consumed by the agent (Layer 8); the gate (Layer 8) reads each tool’s `side_effect` and `quality`.

Workflows it powers

Every workflow is ultimately a small set of tools wired together. The ‘gather’ beats are read tools; the ‘act’ beats are action tools. Building the tool for a capability once lets every workflow and every field reuse it.

Layer 8 — The Agent Core

Status: TO BUILD

What it is

The part that lets the model drive — choosing which tools to call, in what order, to satisfy a trigger — while staying inside the safety rails. This is the leap from ‘smart dashboard with a grounded Q&A box’ to ‘brain’.

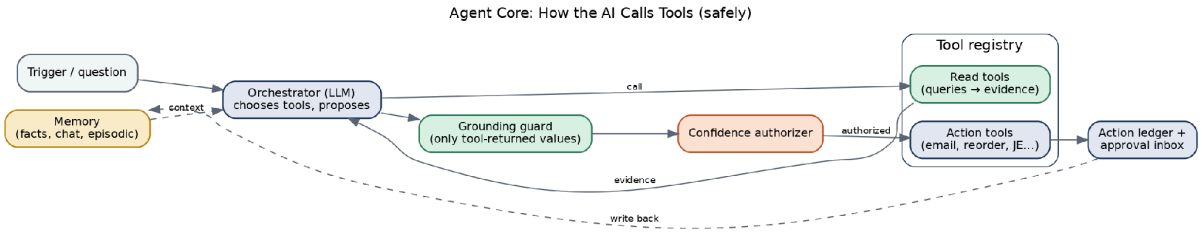

Figure 6 — The agent core: the model proposes tool calls; deterministic code authorises actions.

How it works

7. A trigger (question or event) plus memory context enters the orchestrator (the LLM).
8. The model picks **read tools** to gather what it needs; each returns evidence. It may loop — gather, see results, gather more.
9. It proposes an action (or an answer). The **grounding guard** rejects any stated value not returned by a tool.
10. The **confidence authorizer** — deterministic code — grades the proposed action from the tool quality signals + side-effect class, and routes it: CERTAIN run now, PROBABLE queue for human confirm, UNCERTAIN escalate. Money/regulator always human.
11. The action runs (or waits), and the outcome is written back to memory.

```
# the loop, in essence (raw Anthropic SDK tool-use — no framework needed)
def run_agent(trigger, ctx):
    messages = build_context(trigger, ctx)          # + recent memory (Layer 5)
    while True:
        resp = claude.messages.create(model=tier(trigger),
                  tools=registry.for(ctx.user, trigger), messages=messages)
        if resp.stop_reason != "tool_use":
            return finalize(resp)                    # grounding guard runs here
        for call in resp.tool_calls:
            tool = registry[call.name]
            if tool.side_effect != "read":
                decision = authorizer.grade(tool, call.args, ctx)   # the GATE
                if decision != "CERTAIN":
                    queue_for_human(tool, call.args, decision); continue
            result = runtime.execute(tool, call.args, ctx)          # + ledger
            messages.append(tool_result(call, result))
```

> **Why raw SDK tool-use first, not Google ADK / Microsoft Agent Framework / LangGraph**
> Those frameworks let an AI choose its own actions and orchestrate multi-agent flows — powerful, but they also take ownership of the control loop, which is exactly where your safety lives. The Anthropic SDK already does native tool-calling, so you can build this loop directly and keep the **authorizer as your own code**. Adopt a heavier framework later, *only* when you have many sub-agents handing off to each other (the synapses) and need their orchestration + observability — and even then, the gate stays yours.

How it connects

Consumes the tool registry (Layer 7), memory (Layer 5), and reuses Layer 6’s evidence/validate/gate spine. Emits actions to Layer 9 and events to Layer 10.

Workflows it powers

This is the engine that runs every workflow’s ‘reason → gate’ beats. The same agent core serves all 13 fields; what differs per workflow is only the tools it is given and the trigger that starts it.

Layer 9 — The Action Spine & Human-in-the-Loop

Status: TO BUILD

What it is

The machinery that makes taking real-world actions safe and reversible: a log of everything the brain did, an inbox where ‘confirm-first’ actions wait for a person, and the channels that notify people. The instant the brain can write to the world, this layer must exist.

What to build

- **Action ledger** — every action (proposed, approved, executed, failed) recorded with inputs, the gate decision, and the result. Idempotent (never send the same chase twice) and dry-run-able. The existing sync-run log is the seed of this. PARTIAL
- **Approval inbox** — the UI where PROBABLE actions queue with a preview and confirm/reject buttons, gated by role (Layer 1). TO BUILD
- **Notifications** — email / WhatsApp / in-app, so people are told what needs them and what the brain did. TO BUILD
- **Escalation routing** — UNCERTAIN items go to the right human. TO BUILD

> **The one rule, enforced here**
> Anything that moves money or files with a regulator is **always** routed to a human, regardless of confidence. The ledger + inbox are how that rule becomes real software rather than a promise.

How it connects

Receives actions from Layer 8, records them, and surfaces the queue in Layer 4’s UI. Uses Layer 1 roles to decide who can approve. Notifications close the loop back to the human.

Workflows it powers

Every ‘confirm-first’ and ‘human-led’ workflow lives here: AP payment release, payroll disbursement, tax filing, large quotes, large POs, hiring decisions, legal sign-off, security actions. The middle and bottom rungs of the automation ladder are implemented in this layer.

Layer 10 — Workflows & the Synapse Bus (the actual brain)

Status: TO BUILD

What it is

The layer that turns isolated workflows into one connected, self-running system. A workflow is a named bundle (its trigger, the tools it gathers with, the action it takes, its gate policy). The synapse bus is what lets one event in one field ripple into workflows in other fields — the thing that makes it a brain and not a pile of bots.

What to build

- **Workflow definitions** — declarative: trigger + gather tools + action tool + gate policy. New workflows become data, not new code.
- **Triggers**: schedule (have), manual (have), and **events** (new).
- **The event bus / event log** — when something happens ('invoice overdue'), it is published as an event; workflows subscribe to the events they care about. This is the synapse mechanism.

How a synapse works — the worked example from the concept doc

```
EVENT  invoice.overdue {customer: "Dev Colour", amount: 420000}
  -> Accounts workflow: confirm overdue, worsening pattern        [auto]
  -> Sales workflow:    pause upsell to this customer             [auto]
  -> CRM workflow:      lower account health score                [auto]
  -> Strategy workflow: flag concentration risk in weekly review  [auto]
  ONE event, FOUR fields, no human relay — impossible for separate tools,
  because each tool would hold only one link of the chain.
```

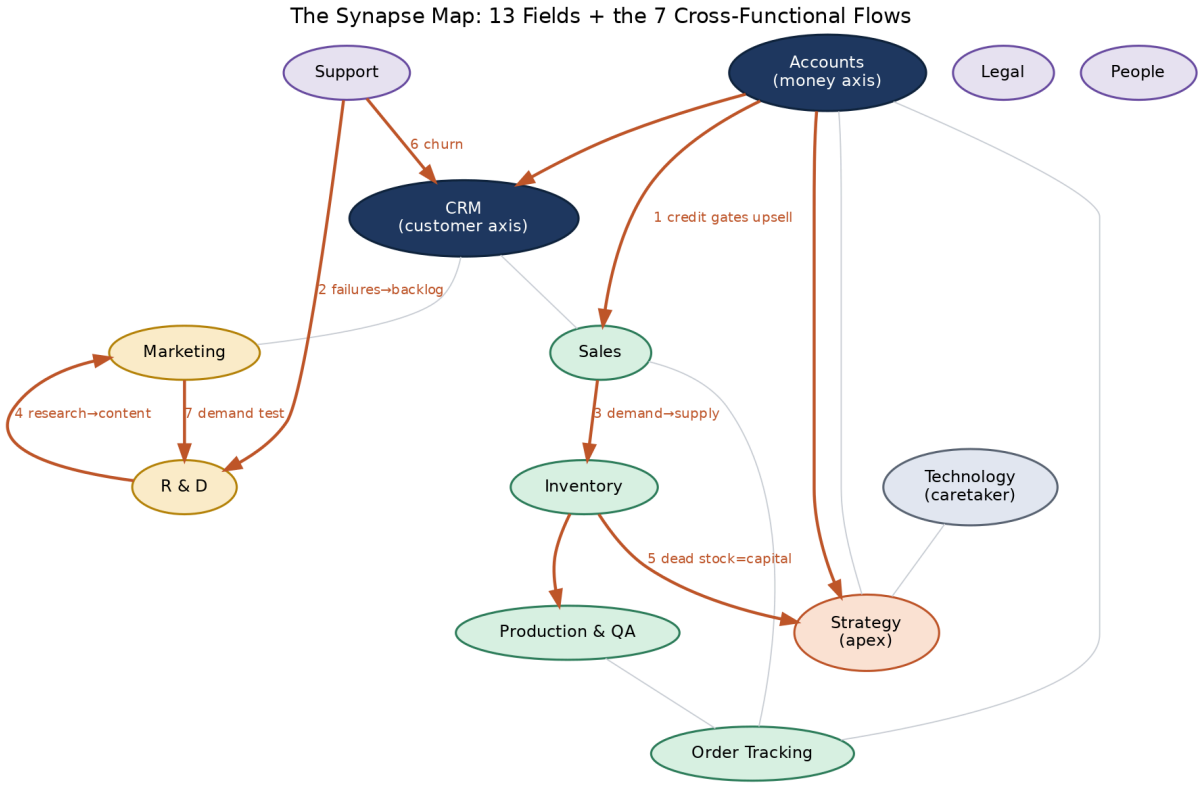

Figure 7 — The synapse map: 13 fields, with Accounts and CRM as the two axes, and the seven cross-functional flows (orange) that justify a single brain.

The seven flagship synapses

#	Cross-functional flow	Why it needs one brain
1	Accounts → Sales	AR/credit gates upsell — don't sell more to a non-payer.
2	Support → R&D / QA	Field failures + requests become the product backlog and quality loop.
3	Sales → Inventory → Production	Pipeline becomes the demand-to-supply plan, no re-keying.
4	R&D → Marketing / Sales	Research becomes battlecards, positioning, content.
5	Inventory + Accounts → Strategy	Dead stock surfaces as trapped working capital.
6	Support + CRM + Accounts → churn	Three signals fuse into one score no single tool can compute.
7	Marketing → R&D	Campaign response is a cheap demand test before building.

The brain graph view (your Obsidian-style idea) is the UI rendering of this map: fields as nodes, synapses as edges, lit up as events flow — both a product feature (let the owner see the brain think) and an ops tool (see what is wired).

How it connects

Sits on top of everything: triggers fire the agent (Layer 8), which uses tools (Layer 7) over canonical data (Layer 3), gated and recorded (Layer 9), remembering outcomes (Layer 5). The event bus is the new connective tissue between workflows.

First synapse to ship

Accounts → Sales — the concept doc names it as the first to prove: the data already exists, it is the lowest-effort, and 'stop extending credit to customers who haven't paid' is a result an owner feels immediately. Prove one synapse end-to-end (with the action gated to a human confirm) and the case for the whole brain makes itself.

Layer 11 — Scale-out & Hardening

Status: **TO BUILD**

Only after one vertical demonstrably works: more adapters (Tally/Busy) and more workflows; cost/performance hardening (model tiering, caching, token budgets); adopting an orchestration framework (ADK / LangGraph / Temporal) if and only if the workflow count and multi-agent handoffs justify it; and deeper observability + eval extended to actions, not just answers.

> **The build sequencing rule (straight from the concept doc)**
> Don’t build the universal brain in the abstract — it becomes a beautiful platform nobody needs. Ship **one vertical deep** (manufacturing, with a live design partner) until it beats the tools it replaced; the reusable bricks and the brain spine fall out as the by-product of having shipped it well. Then point those bricks at a second vertical.

Part II — The Workflows, Field by Field

Every workflow below is one instance of the universal loop. The **Flow** column uses the same five beats: Trigger → Gather → Reason → GATE → Act (or escalate). The **Tier** column places it on the automation ladder: **A** = runs fully automatically, **C** = runs after a human confirms, **H** = human-led, AI assists. The **Layers/notes** column flags what each workflow specially needs.

Reading note: nearly every workflow uses the same stack (ingestion → canonical → tools → agent → gate). The Layers/notes column therefore highlights only the distinctive needs — e.g. an action tool, vector memory, an event trigger, or a hard human gate.

Finance & Accounts

Owns: the books, the cash, and compliance.

Workflow	Flow	Tier	Layers / notes
Invoice & bill capture	doc arrives → OCR → match PO & vendor → GATE clean? → post · else review	C	OCR adapter (L2), action tool
Transaction categorisation	txn → history & rules → propose code + conf → GATE high? → auto-code · else confirm	A	read+write tools
Reconciliation	import → auto-match → clear → GATE unmatched? → exception → human	A	read-heavy
Journal entries	event/schedule → compute → draft JE → GATE rule-based? → post · else approve	C	action tool
Period close	close date → checklist & chase → flag variances → GATE clear? → compile → HUMAN sign-off	C	human gate
Statements & consolidation	close done → assemble → consolidate → GATE balanced? → draft → review → publish	C	action tool
AR aging & collections	past due → aging+history → draft escalation → GATE high-value? → send · else human	A	BUILT base; action tool (email)
Cash application	payment in → match invoices → GATE unambiguous? → apply · else review	A	entity resolution (L3)
Credit & concentration risk	activity → behaviour & exposure → score → GATE threshold? → update · alert human	A	read; durable facts
Accounts payable (3-way match)	bill → match PO↔GRN↔bill → route → GATE release money → HUMAN releases	C	MONEY-gated (L9)
Duplicate-payment / fraud	payment stream → vs patterns → score → GATE suspicious? → hold → human	A	anomaly; human hold
Cashflow forecasting	daily → AR+AP+orders → model → GATE squeeze? → alert · publish	A	read; event
GST returns & ITC recon	period → assemble → reconcile 2A/2B → GATE file → HUMAN files	C	REGULATOR-gated
e-Invoicing & e-way bills	dispatch → build IRN/e-way → GATE valid? → generate · else fix	A	action; source write-back
Payroll computation	cycle → gross→deductions→net → payslips → GATE disburse → HUMAN approves	C	MONEY-gated
Continuous audit & policy	txn/claim → vs policy → GATE breach? → flag · pass	A	monitor

Workflow	Flow	Tier	Layers / notes
NL financial querying	question → query canonical → GATE confident? → answer · else uncertain	A	BUILT TODAY (L6)

Into the brain: Feeds Sales (credit/AR gates upsell) and Strategy (cashflow, working capital). Fed by Order Tracking (dispatch triggers an invoice), Inventory (valuation), and every field that spends.

Sales

Owns: turning interest into revenue.

Workflow	Flow	Tier	Layers / notes
Lead generation & enrichment	inbound → find & enrich → score fit → GATE real? → pipeline · else discard	A	external tools
ICP-fit scoring	lead → vs ICP → score → GATE above? → prioritise · else nurture	A	read
Outreach sequencing	qualified → draft sequence → schedule → GATE high-value? → send · else review	C	action (email)
Pipeline hygiene	scan → stale/incomplete → GATE safe? → auto-update · nudge owner	A	write tool
Deal forecasting	pipeline change → score prob → aggregate → publish → Finance	A	synapse → Finance
Quote & proposal generation	request → product/price/stock/credit → draft → GATE large? → send · else human	C	synapse: reads Inventory+Accounts
Follow-up nudges	no activity N days → draft from history → GATE sensitive? → send · else review	A	action
Win/loss analysis	deal closed → extract reasons → log & route → R&D / Marketing	A	synapse → R&D

Into the brain: Feeds Order Tracking, Production, Inventory (demand), Finance (revenue forecast), Marketing (what converts). Fed by R&D (battlecards), Marketing (leads), Accounts (credit), Inventory (stock), Production (lead times).

Marketing

Owns: creating demand.

Workflow	Flow	Tier	Layers / notes
Content generation & repurposing	brief → draft + variants → GATE brand-sensitive? → publish · else review	C	vector (brand voice)
SEO briefs	topic/gap → research keywords & SERP → draft brief → hand to content	A	external tools
Ad copy & A/B variants	live → variants → A/B → GATE budget change? → scale · spend → human	C	MONEY (budget)
Campaign performance	data → attribute & analyse → report + insight → Sales	A	synapse → Sales
Audience segmentation	refresh → cluster → update segments	A	read
Social scheduling	calendar → draft & queue → GATE risky? → auto-post · else review	A	action
Brand & sentiment monitoring	mentions → classify sentiment → GATE spike? → alert human · log signal	A	event; synapse → R&D

Into the brain: Feeds Sales (qualified leads) and R&D (demand signal). Fed by R&D (positioning), Support (common questions → content), Sales (which segments convert).

Support

Owns: keeping the customers you already won.

Workflow	Flow	Tier	Layers / notes
Tier-1 deflection	ticket → retrieve docs + 360 → draft → GATE confident & simple? → auto-resolve · else agent	A	VECTOR memory (L5)
Ticket routing	ticket → classify topic & priority → GATE clear? → route · else triage	A	read
Sentiment-based escalation	ticket → score sentiment → GATE high risk? → escalate now → human	A	event
Draft replies	agent opens → draft from context → agent edits & sends	H	assist only
KB auto-generation	resolved → extract Q&A → GATE novel? → draft article → review	C	writes vector store
SLA monitoring	clock → track vs SLA → GATE near breach? → alert/escalate	A	event
Ticket-theme analysis	batch → cluster themes → report top issues → R&D / QA	A	synapse → R&D/QA

Into the brain: Feeds R&D, Production & QA, Sales (churn/upsell), CRM. Fed by Order Tracking, CRM, Accounts.

CRM

Owns: the customer relationship — really the customer axis of the brain.

Workflow	Flow	Tier	Layers / notes
Unified customer-360	any interaction → attach to record → update 360 view	A	entity resolution (L3)
Interaction logging	comms event → match to account → GATE clear? → log · else confirm	A	event
Health & churn scoring	refresh → fuse 3 signals → score → GATE at risk? → flag + next-best-action	A	SYNAPSE 6 (fuses 3 fields)
Segmentation	data change → recompute segments → update	A	read
Next-best-action	account state → evaluate → suggest to owner	A	read
Data hygiene & de-dup	scan → detect dupes → GATE safe merge? → auto-merge · else review	A	entity resolution

Into the brain: Ingests from Sales, Support, Order Tracking, Marketing, Accounts; serves all of them.

Order Tracking

Owns: the journey from won deal to delivered goods.

Workflow	Flow	Tier	Layers / notes
Order status tracking	order event → update status → reflect on dashboard	A	event
Milestone & ETA updates	status change → recompute ETA → update	A	reads Inventory+Production
Delay-exception alerts	ETA risk → assess cause → GATE at-risk? → alert ops → human	A	event
Proactive customer notifications	milestone → draft notice → GATE bad news? → send · sensitive → review	C	action (notify)
Fulfillment orchestration	ready → orchestrate dispatch → dispatch → trigger invoice (Accounts)	C	SYNAPSE → Accounts (order-to-cash)

Into the brain: Feeds Support (status), CRM (history), Accounts (invoice trigger), the customer. Fed by Sales, Inventory, Production, Logistics.

Inventory

Owns: the right stock, in the right quantity, at the right time.

Workflow	Flow	Tier	Layers / notes
Stock-health & dead-stock flagging	daily → movement & age → GATE dead? → flag as trapped capital → report	A	SYNAPSE 5 → Strategy
Demand forecasting	sales update → model demand → publish forecast → Production	A	synapse → Production
Auto-reorder at threshold	below point → size PO → GATE large? → auto-PO · else human	C	action; MONEY for large
Negative-stock & integrity monitor	movement → validate balance → GATE invalid? → flag → human	A	data integrity
ABC analysis	periodic → rank A/B/C → update classification	A	read
Stockout prediction	forecast vs stock → detect → GATE imminent? → alert Sales & Production	A	event; synapse

Into the brain: Feeds Sales (available-to-promise), Production (materials), Accounts (valuation), Procurement (reorder). Fed by Sales (demand), Production (consumption/output).

Production & QA

Owns: making the thing, and making it right.

Workflow	Flow	Tier	Layers / notes
Production scheduling	work orders + demand → optimise → GATE conflict? → publish · else planner	C	planning
WIP tracking	station event → update WIP → reflect status	A	event
Predictive maintenance	sensor data → detect degradation → GATE risk? → schedule maint · halt → human	A	telemetry adapter
Vision-based defect detection	unit → inspect image → GATE defect? → divert/flag · trend → QA	A	vision model
Reject-rate & OEE monitoring	production data → compute → GATE drift? → alert → human	A	monitor
Capacity planning	forecast → model capacity → report bottlenecks → Strategy	A	synapse → Strategy

Into the brain: Feeds Inventory (finished goods), Order Tracking (delivery dates), Accounts (COGS/scrap), R&D (yield). Fed by Sales/Orders (demand), Inventory (materials), R&D (specs), Support (field failures).

People Management

Owns: the team — the field that needs the most care.

Workflow	Flow	Tier	Layers / notes
Onboarding & offboarding	hire/exit → checklist & docs → GATE clear? → provision/settle · else HR	C	synapse → Tech (access)
Leave management	request → balance & coverage → GATE safe? → auto-approve · else manager	A	policy
HR policy helpdesk	question → retrieve policy → GATE confident? → answer · else HR	A	VECTOR memory
Attendance & shift management	clock event → compute → GATE anomaly? → record · flag supervisor	A	vertical (shop-floor)
Recruiting logistics	role → draft JD · rank resumes · schedule → DECISION → HUMAN	H	logistics only; never decides
Performance goal-tracking	cycle → aggregate goals & feedback → RATING → HUMAN	H	never auto-rates
Payroll & statutory compliance	cycle → compute pay & statutory → HUMAN disburses/files	C	MONEY/REGULATOR-gated
Learning & development	gap → map → assign path · track	A	read
Engagement & attrition signals	survey → analyse risk → GATE elevated? → surface to HR (private)	A	private handling

Into the brain: Feeds Strategy (capacity, attrition) and Finance (payroll). Fed by every manager. The dangerous parts are deliberately gated to humans.

Legal

Owns: keeping the company safe on paper.

Workflow	Flow	Tier	Layers / notes
Contract review & clause extraction	contract → extract clauses → vs standards → GATE risk? → flag counsel · else summarise	C	VECTOR memory
Contract-lifecycle tracking	calendar → scan obligations → GATE due? → alert owner	A	event
Template generation	request → fill template → GATE non-standard? → draft · else counsel	C	action
Compliance monitoring	reg update → assess impact → GATE action? → alert → human	A	external feed
Document Q&A	question → retrieve corpus → GATE confident? → answer · else counsel	A	VECTOR memory
Risk flagging	new doc → score risk → prioritise → counsel	A	read

Into the brain: Feeds Sales (turnaround), Accounts (payment terms), Strategy (regulatory risk). Fed by Sales, Procurement, People. Sign-off always human.

R & D

Owns: knowing the market and shaping what comes next.

Workflow	Flow	Tier	Layers / notes
Industry & market research	cycle → gather & synthesise → publish brief → Strategy/Marketing	A	external + vector
Competitor teardown	change → analyse features & pricing → update battlecards → Sales	A	synapse → Sales
Trend & patent monitoring	scan → detect signals → GATE material? → brief team	A	external feed
Customer-need mining	tickets + lost deals → cluster needs → rank into backlog	A	SYNAPSE 2 (internal evidence)
Concept generation	need → generate concepts → screen feasibility → shortlist → human	C	assist
Feasibility & cost modelling	concept → model cost & manufacturability → report → decision	C	synapse ← Production

Into the brain: Feeds Marketing (positioning), Sales (battlecards), Strategy (market sizing). Fed by Support (complaints), Sales (lost deals), Production (manufacturability).

Strategy & Planning

Owns: direction — the apex that consumes everything.

Workflow	Flow	Tier	Layers / notes
KPI-vs-target tracking	refresh → actual vs target → GATE off-track? → alert + explain → human	A	reads all fields
Scenario & what-if modelling	question → build scenario → present options → HUMAN decides	H	reads whole model
Competitive landscape	R&D feed → synthesise → brief leadership	A	synapse ← R&D
Capital-allocation analysis	financials + ops → analyse → recommend → human	H	SYNAPSE 5
OKR cascade	goals → cascade to fields → track & report	C	writes targets down
Board & management reporting	cycle → assemble pack → draft → human review	C	read

Into the brain: Fed by every field. Feeds targets back down to all of them. Recommends; a human decides.

Technology

Owns: keeping the brain alive — the caretaker, not a peer field.

Workflow	Flow	Tier	Layers / notes
IT helpdesk	request → retrieve/resolve → GATE routine? → auto-resolve · else IT	A	vector
Access provisioning	HR event → determine access → GATE sensitive? → provision · else human	C	synapse ← People
System & security monitoring	telemetry → detect anomaly → GATE security? → alert → HUMAN (never auto-act)	A	human gate
License management	scan → usage vs licenses → flag waste/renewal → human	A	read
Data-freshness watchdog	pipeline check → last-sync vs SLA → GATE stale? → alert eng + flag dashboard	A	PARTLY BUILT (L2 sync logs)

Into the brain: Serves all fields. Security actions stay human.

Part III — Long-Term Technology Decisions

These are the forks worth deciding deliberately. The rule of thumb running through all of them: keep the safety-critical logic (the gate, scoping, grounding) in our own code, and reach for heavier infrastructure only when scale forces it — not before.

Concern	Recommendation	Why / when to revisit
LLM provider	Anthropic (Claude), with two-tier routing (fast vs smart)	Already integrated; tier by task to control cost. Keep a thin provider abstraction so a second model can slot in.
Agent orchestration	Raw Anthropic SDK tool-use first	Full control of the loop + gate. Revisit ADK / LangGraph / Microsoft Agent Framework only when multi-agent handoffs (the synapses) and cross-run observability justify it.
Tool protocol	In-process tools now; MCP later	MCP adds value only when external/other-app clients must reuse the tools. The tool contract makes the switch cheap.
Event/queue	Postgres-backed event table + worker first	No new infra; fine to thousands of events/day. Move to Redis/Temporal when volume or long-running workflows demand it.
Vector store	pgvector (Postgres extension) first	Keeps one database. Move to a dedicated vector DB only at large corpus scale.
Auth	Keep self-issued JWT; add reset/invite	Roll-your-own is fine at this stage. Consider a managed provider (Clerk/WorkOS) when SSO/SCIM/enterprise demands arrive.
Frontend	Next.js (App Router) — keep	Already powers the BFF security boundary; no reason to change.

Part IV — Build Order & Where We Are

The dependency rule is simple: data before intelligence, intelligence before action, rails before autonomy. The one deliberate exception is the read/dashboard layer in the middle, which delivers value early and doubles as the AI's read surface.

Step	Build	Status
1	Foundations + Identity & tenancy (Layers 0–1)	**BUILT**
2	Ingestion → canonical (Layers 2–3)	Layer 2 **BUILT**; Layer 3 **PARTIAL**
3	Read & present (Layer 4)	**BUILT**
4	Memory, incl. chat-as-context (Layer 5)	**PARTIAL**
5	Reasoning base + eval (Layer 6)	**BUILT**
6	Tools — read + action, the registry (Layer 7)	**TO BUILD**
7	Agent core + confidence authorizer (Layer 8)	**TO BUILD**
8	Action spine + approval inbox (Layer 9)	**TO BUILD**
9	Workflows + synapse bus; first synapse Accounts→Sales (Layer 10)	**TO BUILD**
10	Scale-out: more adapters, workflows, hardening (Layer 11)	**TO BUILD**

Where we are, in one line The entire substrate (Layers 0–3) and the

Step	Build	Status
safe reasoning core (Layer 6) are built; the read layer (4) and memory (5) are partly built. The work ahead is the **action half — Layers 7–10**: tool		

Step	Build	Status
s, the tool-calling agent, the action/approval spine, and the synapse bus. That is the build that turns today’s grounded analytics product into the self		

Step	Build	Status
-running Business Brain.		

Appendix — Glossary

Term	Meaning
Adapter	Code that pulls data from one source (TranzAct, Tally, email) into our system. The moat.
Canonical model	The one source-independent schema (Customer, Invoice, SKU…) every layer above the adapter speaks.
Tool	A function the AI is allowed to call, described so the model can choose to invoke it.
MCP	Model Context Protocol — a standard for exposing tools as a shareable service. Optional; in-process tools work without it.
Evidence / Claim	A retrieved number with an id (evidence); a statement that must cite evidence to assert a number (claim).
Confidence gate / authorizer	Deterministic rules that decide auto / confirm / escalate for each action. The safety core.
Synapse	A cross-functional flow where one event in one field triggers workflows in others. The reason for one brain.
Event bus	The mechanism that publishes events and lets workflows subscribe — how synapses fire.
BFF	Backend-for-Frontend: the Next.js proxy so the browser never touches the backend directly.
JWT	A signed session token; ours, in an httpOnly cookie, carrying the user + brand.

This build manual reflects the os-vinayak codebase read directly from source, mapped against the Business Brain concept and Workflow Reference. Status tags show what exists today; everything else is the planned build.